

Assignment 1

1. Yes there should be pagination. Cursor pagination will be more useful because there might be a large number of chat messages. Cursor pagination also handle adding new messages reliably and can start from specific messages instead of going through and skipping rows.
2. SQL formatting should be handled in the frontend. The backend can return the raw text and frontend can handle any styling.

Assignment 2

1. I would use a JOIN query on Message and not add a field to Conversation. By using JOIN, we avoid any synchronization/duplication issues and any changes are easier to track and maintain. Some disadvantages might be that querying is more complex than reading one column and can be slower for large DBs.

Using a redundant field can be fast and avoids querying everytime to find the last message. However, it has to be updated every time a message is created, deleted, or edited, and it will create duplicate data.

Assignment 3

1. `StubAdapter` is useful for testing without actually calling an LLM. We can use it for CI/CD and unit testing since it always returns a result that we know. It will be cheaper to test using this and it doesn't need an external service.
2. To test invalid SQL, I would just change `StubAdapter` to return an intentionally incorrect SQL query.

Assignment 4

1. In my experiment, few-shot examples didn't have much impact. Both versions had 100% accuracies. few-shot examples might help with more complex queries or ambiguous questions however.
2. If a DB has chinese field names, I would add in the prompt a section to tell the model to never translate any table or column names from the schema.
3. We can store some metadata like the version of SQL, any tables, columns or query types so that when there is a question, we can get examples that are similar and only include those in the prompt.

Assignment 5

1. Yes, it will allow other async requests to continue and `execute_sql` will run in the background. This might require more resource usage and create issues for other parts of the application like DB drivers or connections that might not be thread safe and it will require any exceptions/timeouts or cancelling the request to be handled more carefully.

2. The current system might generate incorrect SQL since everything goes to the SQL generator for now. We can fix this with a query router to classify requests and send them to the right place to be handled. Any data related question should be directed to schema inspection or SQL generation. Unrelated/general questions can be rejected or answered separately. Stuff that is ambiguous will require the model to ask for clarification.
3. It depends on the schema. DDL might increase the accuracy since the LLM will be able to access the exact definition of the table. There are situations where JSON might be better. For example, if there is a large amount of comments or metadata converting everything to DDL might lose that context.
4. Token usage could be stored in a separate table for LLMRequests/usages. This is because a query could involve multiple LLM calls. The new table can reference QueryExecution with many-to-one relation from LLM calls to QueryExecution
5. It might be worth if the results are complex or if users need an explanation. It wouldn't be useful or worth it if the result is simple/an explanation doesn't add value or low cost is a priority.